Assignment 1 Theory Problem Set

DO NOT TAG

Name: Diego Franco
GT Email: dfranco32@gatech.edu

Theory PS Q1. Feel free to add extra slides if needed.

1 Case $i = j$

Figure

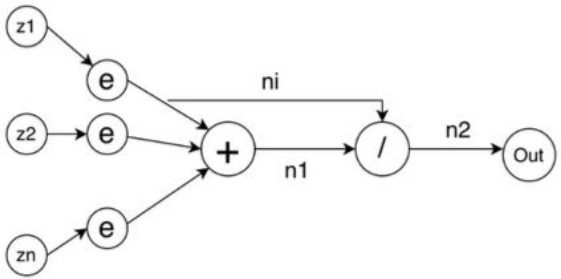


Figure 1: Case $i = j$: two paths from z_i to o (through numerator and through denominator).

Definitions

$$n_i = e^{z_i}$$

$$n_1 = \sum_k n_k$$

$$n2 = \frac{n_i}{n_1} = o$$

Proof

$$\frac{\partial o}{\partial n2} = 1$$

Path 1:

$$\frac{\partial o}{\partial n_{i1}} = \frac{\partial o}{\partial n2}\frac{\partial n2}{\partial n_{i1}} = \frac{1}{n_1}$$

$$\frac{\partial o}{\partial z_{i1}} = \frac{\partial o}{\partial n_{i1}}\frac{\partial n_{i1}}{\partial z_{i1}} = \frac{e^{z_{i1}}}{n_1}$$

Path 2:

$$\frac{\partial o}{\partial n_1} = \frac{\partial o}{\partial n2}\frac{\partial n2}{\partial n_1} = -\frac{n_{i1}}{n_1^2}$$

$$\frac{\partial o}{\partial n_{i2}} = \frac{\partial o}{\partial n_1}\frac{\partial n_1}{\partial n_{i2}} = -\frac{n_{i1}}{n_1^2}1$$

$$\frac{\partial o}{\partial z_{i2}} = \frac{\partial o}{\partial n_{i2}}\frac{\partial n_{i2}}{\partial z_{i2}} = -\frac{n_{i1}e^{z_{i2}}}{n_1^2}$$

Combine both paths:

$$\frac{\partial o}{\partial z_i} = \frac{\partial o}{\partial z_{i1}} + \frac{\partial o}{\partial z_{i2}} = \frac{e^{z_{i1}}}{n_1} - \frac{n_{i1}e^{z_{i2}}}{n_1^2}$$

Where si is softmax(i):

$$\boxed{\frac{\partial o}{\partial z_i} = \frac{e^{z_i}}{n_1} - \frac{e^{z_i}e^{z_i}}{n_1^2} = s_i(1 - s_i)} \quad (1)$$

Theory PS Q1. Feel free to add extra slides if needed.

2 Case $i \neq j$

Figure

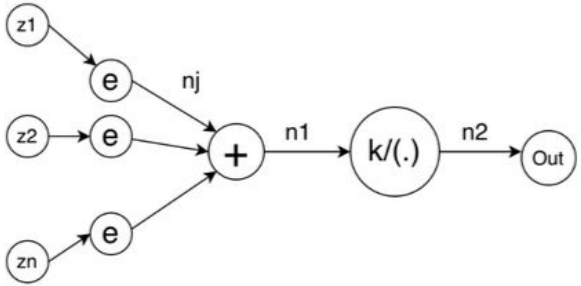


Figure 2: Case $i \neq j$: only the denominator path contributes from z_j to o.

Definitions

$$constant \quad k = e^{z_i}$$

$$n_j = e^{z_j}$$

$$n_1 = \sum_k n_k$$

$$n_2 = \frac{k}{n_1} = o$$

Proof

$$\frac{\partial o}{\partial n_2} = 1$$

$$\frac{\partial o}{\partial n_1} = \frac{\partial o}{\partial n_2}\frac{\partial n_2}{\partial n_1} = -\frac{k}{n_1^2}$$

$$\frac{\partial o}{\partial n_j} = \frac{\partial o}{\partial n_1}\frac{\partial n_1}{\partial n_j} = -\frac{k}{n_1^2}1$$

Where sj is softmax(j)

$$\boxed{\frac{\partial o}{\partial z_j} = \frac{\partial o}{\partial n_j}\frac{\partial n_j}{\partial z_j} = -\frac{ke^{z_j}}{n_1^2} = -\frac{e^{z_i}e^{z_j}}{n_1^2} = -s_i s_j} \quad (2)$$

Theory PS Q1. Feel free to add extra slides if needed.

Final Result

From Eq(1) $i = j$ it obtained $\frac{\partial s_i}{\partial z_i} = s_i(1 - s_i)$, and from Eq(2) $i \neq j$ it obtained $\frac{\partial s_i}{\partial z_j} = -s_i s_j$. These two expressions can be written in a single compact form using the Kronecker delta δ_{ij} as follows:

$$\delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

$$\boxed{\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j)} \quad (3)$$

Check: If $i = j$, then $\delta_{ij} = 1$ and Eq(3) gives $s_i(1 - s_i)$. If $i \neq j$, then $\delta_{ij} = 0$ and Eq(3) gives $-s_i s_j$.

Theory PS Q2. Feel free to add extra slides if needed.

AND

Proof

Choose

$$w_1 = 1, \quad w_2 = 1, \quad b = -2.$$

Then $w_1x_1 + w_2x_2 + b = x_1 + x_2 - 2$, so:

$$(0,0): \quad 0+0-2 = -2 < 0 \;\Rightarrow\; f(0,0) = 0$$

$$(1,0): \quad 1+0-2 = -1 < 0 \;\Rightarrow\; f(1,0) = 0$$

$$(0,1): \quad 0+1-2 = -1 < 0 \;\Rightarrow\; f(0,1) = 0$$

$$(1,1): \quad 1+1-2 = 0 \geq 0 \;\Rightarrow\; f(1,1) = 1$$

Therefore AND is representable by a single linear threshold neuron.

OR

Proof

Choose

$$w_1 = 1, \quad w_2 = 1, \quad b = -1.$$

Then $w_1x_1 + w_2x_2 + b = x_1 + x_2 - 1$, so:

$$(0,0): \quad 0+0-1 = -1 < 0 \;\Rightarrow\; f(0,0) = 0$$

$$(0,1): \quad 0+1-1 = 0 \geq 0 \;\Rightarrow\; f(0,1) = 1$$

$$(1,0): \quad 1+0-1 = 0 \geq 0 \;\Rightarrow\; f(1,0) = 1$$

$$(1,1): \quad 1+1-1 = 1 \geq 0 \;\Rightarrow\; f(1,1) = 1$$

Therefore OR is representable by a single linear threshold neuron.

Theory PS Q3. Feel free to add extra slides if needed.

XOR

Proof (Intuition)

If we plot $z = x_1 + x_2 + b$, there is no way to have a linear plane that evaluates to 0 on points $(0,0)$ and $(1,1)$, while having $(0,1)$ and $(1,0)$ different from 0. This is impossible in Euclidean geometry.

Proof (Contradiction)

Assume XOR can be represented by a single linear threshold neuron with some (w_1, w_2, b).

$$(0,0) \to 0: \quad 0 + 0 + b < 0 \;\Rightarrow\; b < 0 \tag{1}$$

$$(0,1) \to 1: \quad 0 + w_2 + b \geq 0 \tag{2}$$

$$(1,0) \to 1: \quad w_1 + 0 + b \geq 0 \tag{3}$$

$$(1,1) \to 0: \quad w_1 + w_2 + b < 0 \tag{4}$$

Add the inequalities from $(0,1)$ and $(1,0)$:

$$(w_2 + b) + (w_1 + b) \geq 0 \;\Rightarrow\; w_1 + w_2 + 2b \geq 0 \;\Rightarrow\; w_1 + w_2 \geq -2b \tag{5}$$

Since $b < 0$, define $c = -b > 0$. Then

$$w_1 + w_2 \geq 2c \tag{6}$$

From the condition for $(1,1) \to 0$:

$$w_1 + w_2 + b < 0 \;\Rightarrow\; w_1 + w_2 < -b \;\Rightarrow\; w_1 + w_2 < c \tag{7}$$

The last two inequalities, Eq(6) and Eq(7), contradict each other. Therefore XOR cannot be represented by a single linear threshold neuron.

Assignment 1 Paper Review

DO NOT TAG

Name: Diego Franco
GT Email: dfranco32@gatech.edu

Paper: Understanding deep learning requires rethinking generalization

- The paper dives into detail on a set of experiments to test whether the understanding at that point about generalization was correct. There are 3 main important conclusions, 1) classical capacity-based explanations suggested big capacity should overfit unless controlled, the tests in the paper suggest this is not the case, 2) a sufficiently wide two layer ReLU network can fit any labels on N data training points, 3) regularization does not reliably prevent overfitting random labels.

  The first point was proven by shuffling the training labels from a dataset and also from sampling labels from random distributions and showing the models were able to fit the assigned training labels. Regarding the second point is that overparameterized networks can have enough finite-sample expressivity to fit any labeling, where they provide a theorem "There exists a two-layer neural network with ReLU activations and 2n + d weights that can represent any function on a sample of size n in d dimensions", however this expressivity does not directly correlate to the generalization performance. Note this makes classical measures such as VC and Rademacher not very informative in this regime, where it would show high capacity whether the labels are random or not.

  The authors showed how regularization does not affect learning training data, the third point above, where different models with and without regularization were tested. The results showed that the training data can fit equally well with or without regularization. Regarding generalization, the regularization method that performed best were the ones that introduced new information such as data augmentation (via known invariances) compared to the rest e.g. weight decay. They conclude that explicit regularization can help but is not necessary or sufficient to improve test error. Also, from the SGD perspective the nature of the algorithm prefers the smallest norm solution that fits the data, this preference may help generalize better. Furthermore all interpolating minima share essentially the same Hessian structure, so local curvature is unable to discriminate from zero training error solutions to know which one will generalize better, as opposed to SGD where minimum norm picks up a particular one.

Personal view of the paper:

- One of the key parts of the paper is providing an empirical demonstration to generalization regarding model capacity, having a deeper understanding about it allows for better training pipelines. I liked how they used random sampling and showed how any sufficiently wide neural network model can fit the training labels, yet not generalizing well due to lack of structure.

  The random label training can be compared with a stochastic process, in that when the model fits any data it could be viewed to some extent to a snapshot of a Brownian motion, and when forecasting it behaves similarly to a next time window in a Brownian motion (or similarly N-body problem). It would be very interesting to dig deeper into the differential stochastic equations when a stochastic process transitions from diffusion to drift and comparing how a neural network transitions from low to high accuracy, it might offer new insights about the random forecast behavior in the network.

  Likewise, research from Halverson et al. (1) does the comparison between a neural network and an effective quantum field theory, where these fields behave randomly with many degrees of freedom, yet emergent behavior leads to lower entropy coarse grain reality. In Quantum Field Theory (QFT) renormalization averages out the uncertainty simplifying the calculations, and it would be interesting to research how a neural network that is able to generalize well applies similar techniques to renormalization in QFT, and also explore the blurred line between noise output and a well generalized model.

1) https://arxiv.org/abs/2008.08601

Paper specific Q1. Feel free to add extra slides if needed.

- Neural networks can learn more generalizable representations when there are meaningful patterns in the data, consider having random labels where the correlation between inputs and outputs is almost zero, that means a model would need to learn N degrees of freedom, yet the next sample is not correlated to anything learned before in a meaningful way. This is in contrast when there is pattern in the data, at that point the model is able to capture the lower degrees of freedom that describe it, and therefore forecast more accurately.

  Also, regularization is precisely trying to reward features that survive noise, trying to capture structure in the dataset. Furthermore, a model that learns a structured dataset is one function of many in the hypothesis class, where some may perform better than other ones depending on out of sample data, therefore learning a structure is affected by the quality of the input data set, even when is not random, so that the right degrees of freedom are captured.

Paper specific Q2. Feel free to add extra slides if needed.

- This finding aligns well with my current understanding of machine learning and generalization. However I was not aware of the theorem in the paper stating that any sufficiently wide 2 layer neural ReLU network with enough parameters relative to N training points can fit any dataset.

  Also, after this paper validation feels even more important, and highlights the importance of using and researching the right statistical theoretical tools and metrics for the network, since regular ones do not necessarily work well. Regarding regularization clarifies necessity vs sufficiency where it can help but may not be enough for good generalization.

  Lastly it gave me a better view of SGD not only implementing model training but also helping with implicit regularization.

Assignment 1 Writeup

DO NOT TAG

Name: Diego
GT Email: dfranco32@gatech.edu

Two-Layer Neural Network

DO NOT TAG

Note

All of the experiments were run with a copy of main.py called exp_main.py, it has custom outputs and adds regularization values as title in charts:

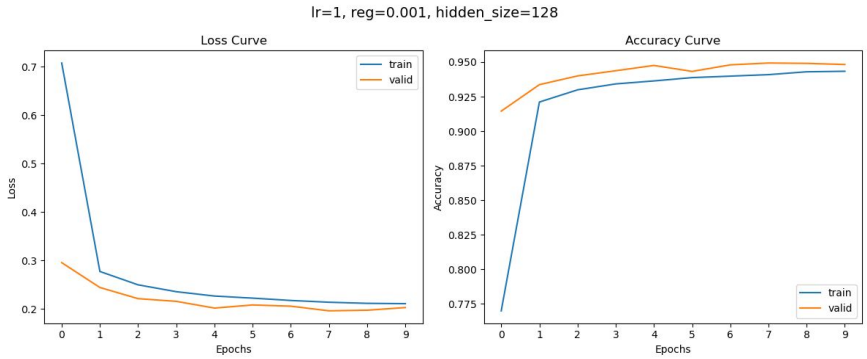
```
$python3 exp_main.py --config configs/config_exp.yaml
```

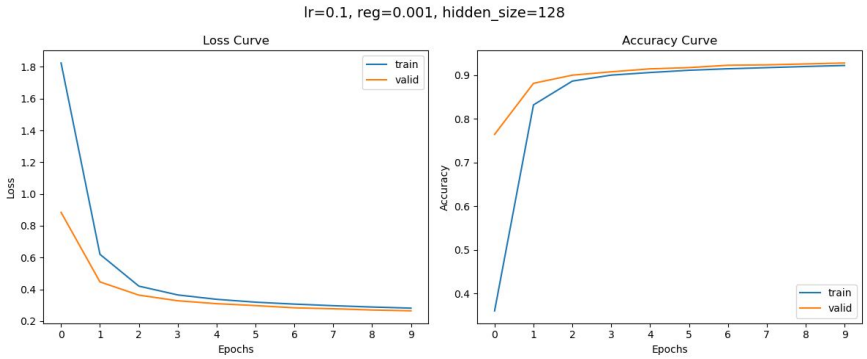
1. Learning Rates

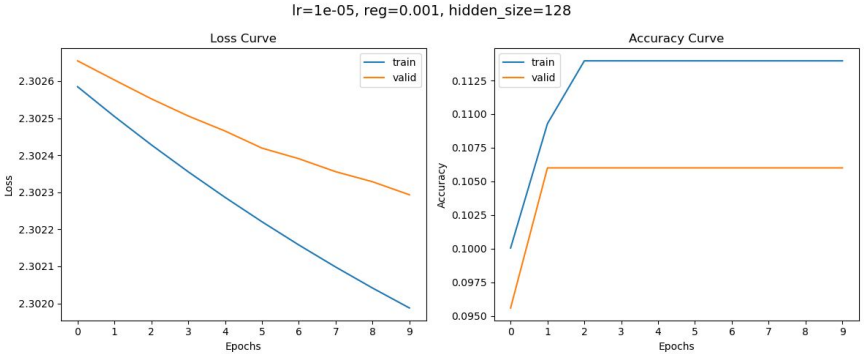
Tune the learning rate of the model with all other default hyper-parameters fixed. Fill in the table below:

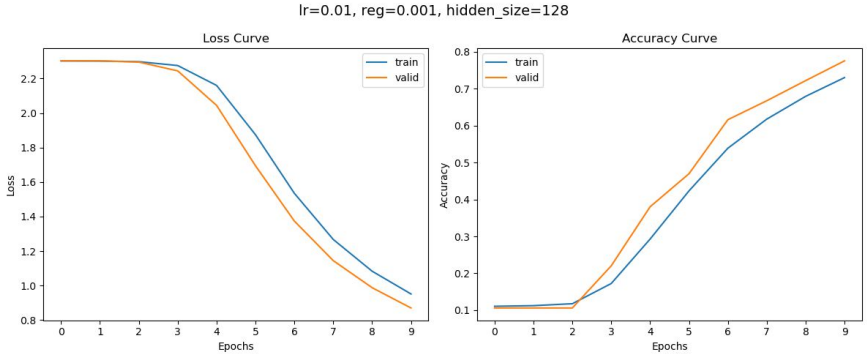
	lr=1	lr=1e-1	lr=5e-2	lr=1e-2
Training Accuracy	0.9434	0.9217	0.1140	0.7301
Test Accuracy	0.9489	0.9250	0.1135	0.7652

1. Learning Curve, only lr changed rest default









1. Learning Rates

Describe and Explain your findings:

- The learning rate is the step size to walk the loss landscape, having a large value allows to explore more territory, such as better local minimums. In the obtained results, the largest learning rate was 1.0, and in this case it worked best as it allowed to find better minimums, as the step became smaller it tend to be stuck in a local area in the loss landscape (based on budgeted training samples), and therefore the loss was worse as learning rate was smaller.
- Note, It may not be always the case that using a larger learning rate finds better values, as it can also skip to other parts of the landscape fast without exploring enough the current local area where a good minimum could exist.
- For lr = 1 the training loss starts around 0.7 goes down fast and then keeps going down slowly once it found a good minimum, where validation starts at 0.3 immediately, going down overall and ramping up again around 8th epoch showing overfitting. The validation error is much lower than training on start since the training started with random weights, where validation started with trained weights, therefore having less error, also it could be help the validation set being a smaller set of samples compared to larger training set that compromises more challenging images to be learned.
- The accuracy flattens for lr = 1, 0.1, 1e-5, since accuracy is a coarse grained value, meaning you need to lower the error considerably in order to add another match, this band gap gets reflected by a flattened behavior in training after some epochs in training, in validation almost similar behavior is seen, however is wiggly due to sampling noise, in training all the same values were learned many times optimizing for those values only, in validation new values are sampled.
- lr = 0.1 still shows similar behavior as 1 with slightly more noise and less accuracy, reflecting the step size is good enough to explore new minimums.
- lr = 1e-5 is the worst performance as the step is very small gets stuck in a local area with high error and never reaching a stable value.
- lr = 0.01 is almost at the intersection of being able to find a better minimum and getting stuck in local area, it can be seen the training and validation were similar from epoch 0 to 5, the finally found a good minimum and loss started going down.

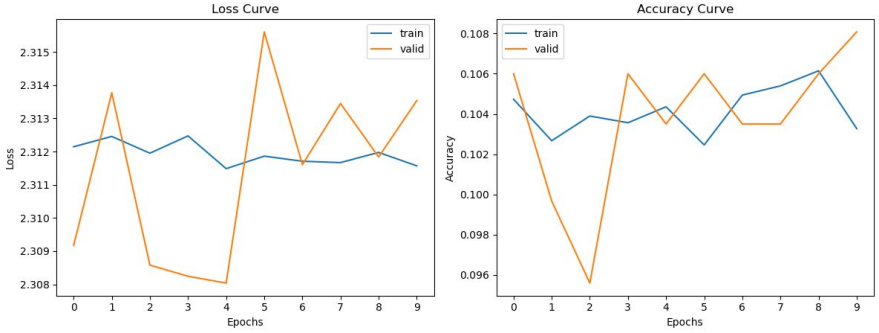
2. Regularization

Tune the regularization coefficient of the model with all other default hyper-parameters fixed. Fill in the table below:

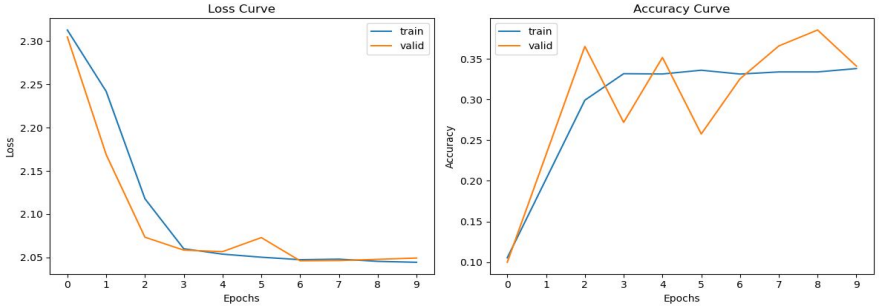
	alpha=1	alpha=1e-1	alpha=1e-2	alpha=1e-3	alpha=1e-4
Training Accuracy	0.1033	0.3380	0.8840	0.9216	0.9295
Validation Accuracy	0.1081	0.3410	0.8940	0.9259	0.9367
Test Accuracy	0.1028	0.3811	0.8930	0.9247	0.9335

2. Regularization, only reg changes, rest default

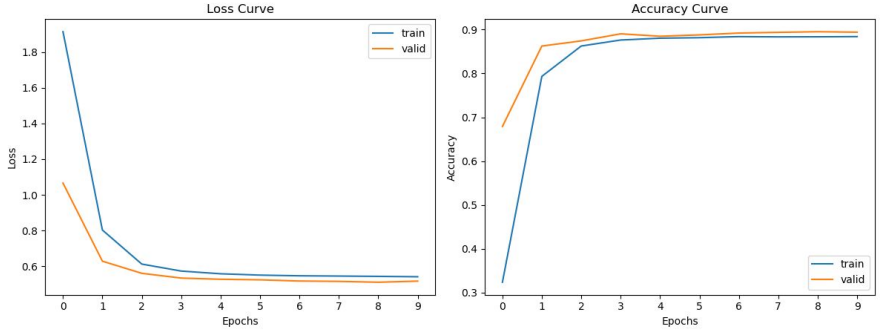
lr=0.1, reg=1, hidden_size=128



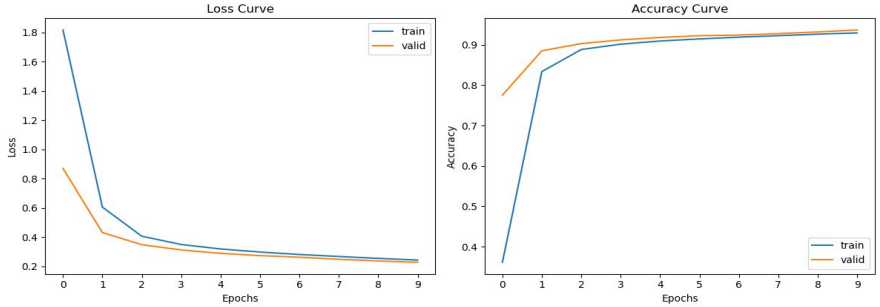
lr=0.1, reg=0.1, hidden_size=128



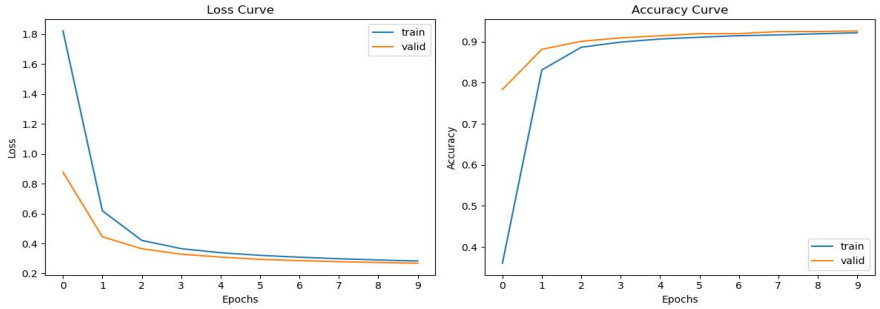
lr=0.1, reg=0.01, hidden_size=128



lr=0.1, reg=0.0001, hidden_size=128



lr=0.1, reg=0.001, hidden_size=128

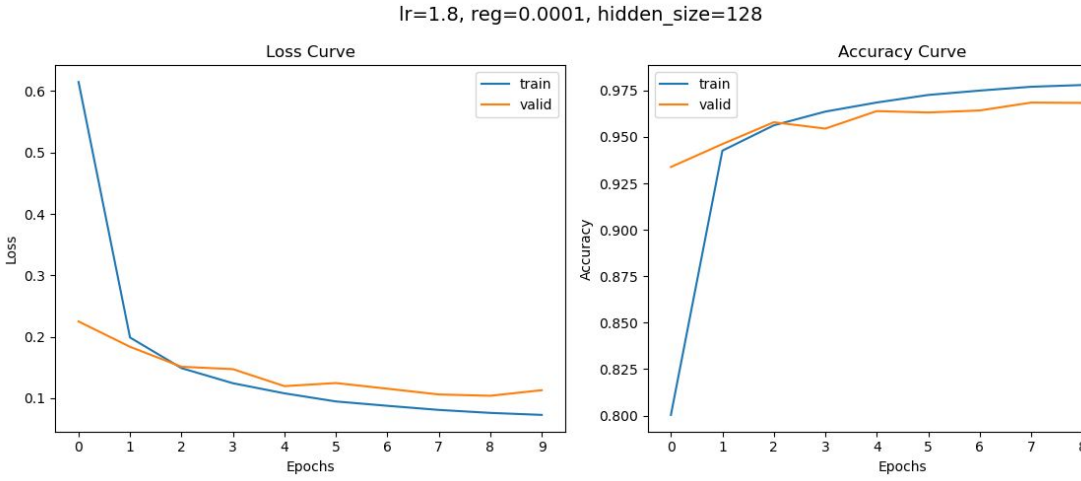


2. Regularization

- The regularization affects how easily a weight is updated, when regularization is high only strong gradient changes lead to changes in weight, otherwise it'll be close to zero.
- It can be seen when reg=1, some of the weights are close to zero and therefore the error and noise are high, since the model has not learned any meaningful training data. i.e. the model is underfitting.
- As regularization is decreased more weights are set to meaningful values and the model starts learning better the input data.
- Similar behavior as lr regularization is seen in terms of accuracy being flattened, where the coarse granularity to change the accuracy needs a larger step in error loss.
- The difference in error from reg=0.001 and 0.0001 is very small, suggesting this value has reached the argmin_reg(loss), also the validation keeps going lower slightly for higher epochs, still not overfitting the data.

3. Hyper-parameter Tuning

	lr = 1.8, reg=0.0001, hidden_size=128
Training Accuracy	0.9796
Validation Accuracy	0.9634
Test Accuracy	0.9678



3. Hyper-parameter Tuning

- I changed first the hidden size and noticed that size=1 had accuracy 0.2, size=5 accuracy 0.83, what tells me the model is almost linear, where not just one linear regression is enough, but few ones correctly forecast it.
- The simplicity of the model suggests that having larger lr is safe, as it may not have many local minimas, also by checking the first experiment lr results, I kept increasing the lr as it seemed it had not reached the max value. Running a large learning rate could lead to preventing the gradient descent converging, where I tested a very large value of 10 leading to 0.7 accuracy. I keep reducing the values until it got a max around 1.8. Having a larger lr also can help it escaping local minima.
- Lastly based on second experiment the best reg found was in order 1e-4, suggesting all of the weights contribute meaningfully to the model. There is a bit of contradiction in having a simple model per small hidden size experiments, but yet requiring all the weights (low reg), it is the balance of these two knobs that allows for better accuracy. I also tested lr=1.8, hidden size=128 and higher reg=0.1 and lead to very small training accuracy = 0.1, suggesting both reg must be very small and lr high.
- It can be seen from the figure the validation starts overfitting at epoch 8 (from some values before it went slightly up be kept going down further overall).
- The accuracy in validation is a bit wiggly around the training one, due to out of sample training data, i.e. model has not learned those values exactly. In addition, the accuracy reaches it maximum at epoch 8 as described above.